

Clasificación de jets de partículas subatómicas mediante Machine Learning con el conjunto de datos JetClass

Trabajo Fin de Grado
Curso 2023–2024

Ana Fuentes Rodríguez

Índice

1. Introducción
2. Materiales
3. Metodología
4. Resultados
5. Conclusiones

Introducción

JETCLASS

- ▶ Conjunto de datos a gran escala
- ▶ Creado por Qu et al
- ▶ Contenido:
 1. 100M jets de entrenamiento
 2. 20M jets de evaluación.
 3. 5M jets de validación

Introducción

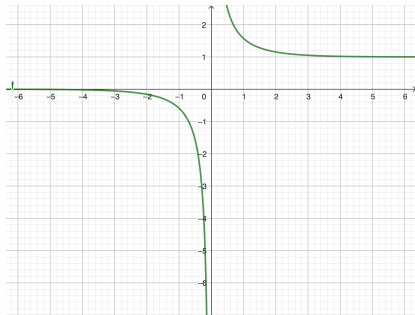
ELEMENTOS DE UNA RED

- ▶ Capas neuronales
- ▶ Pesos sinápticos
- ▶ Ley de programación
- ▶ Función de activación:
 1. Sigmoide
 2. Tanh
 3. ReLU
 4. Leaky ReLU
 5. Softmax

Introducción

► Función Sigmoide

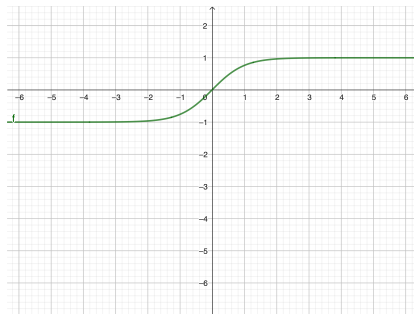
$$f(x) = \frac{1}{1 - e^{-x}} \quad (1)$$



Introducción

► Función Tangente hiperbólica (Tanh)

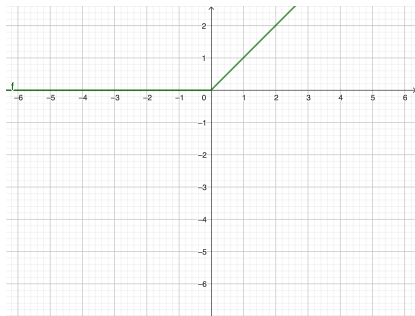
$$f(x) = \frac{2}{1 + e^{-2x}} - 1 \quad (2)$$



Introducción

► Función ReLU

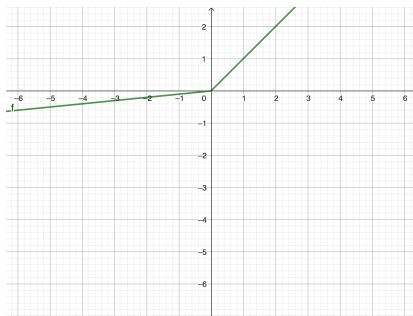
$$f(x) = \max(0, x) \begin{cases} 0 & \text{si } x < 0 \\ x & \text{si } x \geq 0 \end{cases} \quad (3)$$



Introducción

► Función Leaky-ReLU

$$f(x) = \begin{cases} \alpha x & \text{si } x \leq 0 \\ x & \text{si } x > 0 \end{cases} \quad (4)$$



Introducción

► Función Softmax

$$f(Z)_j = \frac{e^{Z_j}}{\sum_{K=1}^K e^{Z_K}} \quad (5)$$

CONJUNTO JETCLASS

- ▶ Contenido:
 1. 100M jets de entrenamiento (empleado: 50%)
 2. 20M jets de evaluación
 3. 5M jets de validación
- ▶ 10 categorías para clasificar.
- ▶ Características de jets, partículas y características auxiliares.
- ▶ Almacenado en archivos con formato `.root`

CONTENIDO DE LOS ARCHIVOS .ROOT

```
1 import uproot
2
3 nombre_fichprueba="HToBB_000.root"
4 fichprueba=uproot.open(nombre_fichprueba)
5 arbol=fichprueba["tree"]
6
7 arbol.show()
```

IMPORTAR LIBRERÍAS

- ▶ Tensorflow
- ▶ De sklearn.preprocessing: StandardScaler
- ▶ numpy
- ▶ uproot
- ▶ os

LEER ARCHIVOS

```
1 def leer_archivo(camino):
2     arch = uproot.open(camino)
3     arb = arch["tree"]
4
5     cesc = [
6         arb["jet_phi"], arb["jet_energy"]
7     ]
8
9     cvec = [
10         np.sum(i) for i in arb["part_px"].array(
11             library="np"),
12         np.array([np.sum(i) for i in arb["part_py"]
13             ].array(library="np")])
14     ]
15
16     carac = np.column_stack(cesc + cvec)
17
18     etiq = np.stack([
19         arb["label_Hcc"], arb["label_Hgg"]], axis
20         =-1)
```

IMPLEMENTACIÓN, COMPILACIÓN Y ENTRENAMIENTO DEL MODELO

```
1 modelo = models.Sequential([
2     layers.Input(shape=(train_data.shape[1],)),
3     layers.Dense(10, activation="relu"),
4     layers.Dropout(0.5),
5     layers.Dense(10, activation="relu"),
6     layers.Dropout(0.5),
7     layers.Dense(10, activation="relu"),
8     layers.Dropout(0.5),
9     layers.Dense(10, activation="softmax")
10 ])
11
12 modelo.compile(optimizer="adam",
13               loss="categorical_crossentropy",
14               metrics=["accuracy"])
15 historial=modelo.fit(datos_entreno, etiq_entreno,
16                     epochs=5, datos_val=(datos_val, etiq_val))
```

Resultados

VARIACIÓN DEL NÚMERO DE NEURONAS Número de capas ocultas 3 y función de activación ReLU

# de neuronas	Pérdida	Desviación	Tiempo (minutos)
10	1,19	0,186	42,5
20	0,433	0,104	42,82
30	0,152	0,0160	44,0
40	0,104	0,0102	44,3
50	0,103	0,0164	49,4
60	0,102	0,0148	50,9

Resultados

VARIACIÓN DEL NÚMERO DE CAPAS OCULTAS

Número de neuronas 40 y función de activación ReLU

# de capas	Pérdida	Desviación	Tiempo (minutos)
3	0,104	0,0102	44,3
4	0,220	0,056	48,4
5	0,448	0,174	52,5
6	0,468	0,187	55,5
7	0,618	0,242	57,3

Resultados

VARIACIÓN DE LA FUNCIÓN DE ACTIVACIÓN

Número de neuronas 40 y número de capas ocultas 3

Función de activación	Pérdida	Desviación	Tiempo (minutos)
Sigmoide	0,130	0,0138	52,5
Tanh	0,145	0,00590	48,3
ReLU	0,104	0,0102	44,3
Leaky ReLU	0,117	0,00583	48,9
Softmax	0,105	0,210	51,6

EVALUACIÓN DEL MODELO FINAL

	Función pérdida	Desviación típica
Entrenamiento	0,104	0,0102
Evaluación	0,151	0,0162

Resultados

COMPARACIÓN CON PARTICLENET Y PART

Modelo	Precisión
ParticleNet	85,6%
ParT	88,9%
Red neuronal	94,4%

Conclusions

- ▶ Output layer: 10 neurons and Softmax activation function
- ▶ 3 hidden layers: 40 neurons and ReLU activation function
- ▶ Hidden layers with Dropout function